

Cursor Export - CS 182 Homework 4

Question 5: Designing 2D Filters

Exported on 10/2/2025 at 22:30:57 PDT from Cursor (1.7.28)

User

Write code to implement the `average_filtering` function with the `filter_size` parameter

Prompting Cursor to implement the first TODO in the `HandDesignFilters.ipynb`

Cursor

I'll open the notebook to locate the `averaging_filtering` function and identify the exact cell index so I can implement it directly.

I'll implement the averaging kernel using `np.ones((filter_size, filter_size)) / (filter_size * filter_size)` and convolve it with the input image in the existing function.

```
def averaging_filtering(image, filter_size=3):
#####
# TODO: Implement the averaging filter with the given filter size. #
# Hint: You can use np.ones #
#####
#                                     END OF YOUR CODE #
#####
output = ndimage.convolve(image, kernel)
return output
```

Original code
(boxed in
blue)

->

```
def averaging_filtering(image, filter_size=3):
    kernel = np.ones((filter_size, filter_size), dtype=float) / (filter_size * filter_size)
    output = ndimage.convolve(image, kernel)
    return output
```

Cursor's
edits
(boxed
in green)

Implemented averaging filter using a normalized `np.ones` kernel and convolving with `ndimage.convolve`.

- Next step: run the notebook cell that builds `avg_images` to visualize results.

User

Prompting Cursor to implement the second TODO in the notebook

Implement the `edge_detecting` function using a 3x3 Laplacian kernel:

```
$
\left[
\begin{array}{ccc}
0 & 1 & 0 \\
1 & -4 & 1 \\
0 & 1 & 0
\end{array}
\right]
$
```

In general, I have noticed that AI tools typically struggle at editing Jupyter notebooks, especially when number of cells in the notebooks grow very large. I've had experiences where the LLM will think, try to make an edit, perform an action that makes no changes, try to grep for a certain cell in the notebook, fail to find it, and get stuck in a loop. But with this relatively small notebook and a clear TODO, Cursor implemented the task in a single shot.

Cursor

I'll implement the `edge_detecting` function using the 3x3 Laplacian kernel as specified. Let me update the function in the notebook.

```
def edge_detecting(image):
    #####
    # TODO: Implement the Laplacian derivative filter. #
    #####
    #####
    #                               END OF YOUR CODE #
    #####
    output = ndimage.convolve(image, kernel)
    return output
```

->

```
def edge_detecting(image):
    kernel = np.array([[0, 1, 0],
                       [1, -4, 1],
                       [0, 1, 0]], dtype=float)
    output = ndimage.convolve(image, kernel)
    return output
```

Implementing the Laplacian kernel was also very easy for Cursor, since the instructions were pretty straightforward. I'm not sure why Cursor insists on deleting all the TODOs and the END OF YOUR CODE block though.

Perfect! I've implemented the `edge_detecting` function using the exact 3x3 Laplacian kernel you specified:

```
[[0, 1, 0],
 [1, -4, 1],
 [0, 1, 0]]
```

The function creates this kernel as a numpy array and convolves it with the input image using `ndimage.convolve`. This Laplacian kernel will detect edges by computing the discrete Laplacian of the image, highlighting areas where the intensity changes rapidly (edges).

You can now run the cell to see the edge detection results compared to the original image!

```
[[0, 1, 0],
 [1, -4, 1],
 [0, 1, 0]]
```

I appreciate the explanation Cursor added to describe how the Laplacian kernel functions. I think providing motivation behind the Laplacian kernel helps re-frame this as an interesting application of math/convolutions rather than an abstract, coding problem